



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251182
Nama Lengkap	Michael Dylan
Minggu ke / Materi	15 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU 1 (40%)

15.3.1 Pengertian Rekursif

Rekursif merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk menyelesaikan suatu permasalahan dengan cara mengulang proses melalui fungsi yang memanggil dirinya sendiri. Secara sederhana, rekursif dapat diartikan sebagai proses pengulangan yang dilakukan oleh sebuah fungsi secara otomatis. Ketika fungsi dijalankan, fungsi tersebut dapat memanggil dirinya kembali dengan nilai parameter yang berbeda sehingga proses akan terus berjalan sampai mencapai kondisi penghentian. Oleh karena itu, rekursif memiliki hubungan yang sangat erat dengan konsep pengulangan atau looping. Dalam dunia pemrograman, rekursif banyak digunakan karena dapat membuat penulisan program menjadi lebih singkat, rapi, dan mudah dipahami pada kasus tertentu. Beberapa contoh permasalahan yang sering menggunakan rekursif adalah perhitungan faktorial, deret Fibonacci, pencarian pada struktur data tree, dan algoritma tertentu seperti divide and conquer. Dengan menggunakan rekursif, proses penyelesaian masalah dapat dibagi menjadi bagian-bagian kecil yang memiliki bentuk penyelesaian yang sama.

Walaupun terlihat sederhana, penggunaan fungsi rekursif harus dilakukan dengan hati-hati. Hal ini karena fungsi rekursif dapat berjalan terus-menerus apabila tidak memiliki kondisi penghentian yang jelas. Jika sebuah fungsi terus memanggil dirinya sendiri tanpa berhenti, maka program dapat mengalami infinite loop atau pengulangan tanpa akhir. Kondisi tersebut dapat menyebabkan penggunaan memori menjadi sangat besar sehingga program menjadi lambat, hang, bahkan crash.

Pada fungsi rekursif, terdapat proses yang disebut sebagai pemanggilan berulang. Setiap kali fungsi memanggil dirinya sendiri, sistem akan menyimpan proses tersebut di dalam memori. Semakin banyak pemanggilan yang terjadi, maka semakin besar juga penggunaan memorinya. Oleh sebab itu, rekursif harus dirancang dengan kondisi penghentian yang tepat agar proses dapat selesai sesuai tujuan program.

Sebuah fungsi rekursif umumnya memiliki dua bagian penting, yaitu Base Case dan Recursive Case. Kedua bagian ini sangat penting karena menentukan apakah fungsi dapat berjalan dengan benar atau tidak.

BASE CASE

Base Case adalah kondisi yang digunakan untuk menghentikan proses rekursif. Bagian ini merupakan titik akhir dari pengulangan fungsi. Ketika kondisi Base Case terpenuhi, fungsi tidak lagi memanggil dirinya sendiri dan proses rekursif akan berhenti. Tanpa adanya Base Case, fungsi akan terus berjalan tanpa akhir sehingga dapat menyebabkan error pada program. Oleh karena itu, Base Case merupakan bagian paling penting dalam rekursif.

Contoh sederhana Base Case:

```
if n == 0:  
    return 1
```

Pada contoh tersebut, ketika nilai n sama dengan 0, fungsi akan berhenti dan mengembalikan nilai tertentu.

RECURSIVE CASE

Recursive Case adalah bagian dimana fungsi memanggil dirinya sendiri secara berulang. Pada bagian ini terdapat proses utama dari rekursif. Biasanya nilai parameter akan berubah secara bertahap menuju kondisi Base Case.

Contoh sederhana Recursive Case:

```
return n * faktorial(n - 1)
```

Pada bagian tersebut, fungsi faktorial() memanggil dirinya sendiri dengan nilai $n - 1$ sampai akhirnya mencapai kondisi berhenti.

CARA KERJA REKURSIF

Cara kerja rekursif sebenarnya hampir mirip dengan proses pengulangan biasa, namun dilakukan melalui pemanggilan fungsi. Ketika fungsi dipanggil, program akan menjalankan instruksi di dalam fungsi tersebut. Jika di dalamnya terdapat pemanggilan fungsi yang sama, maka program akan masuk kembali ke fungsi tersebut hingga mencapai Base Case. Setelah mencapai Base Case,

proses akan kembali satu per satu ke pemanggilan sebelumnya sampai semua proses selesai. Mekanisme ini disebut dengan stack atau tumpukan proses.

Sebagai contoh pada perhitungan faktorial:

```
def faktorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * faktorial(n - 1)  
print(faktorial(5))
```

Penjelasan prosesnya adalah sebagai berikut:

- faktorial(5) memanggil faktorial(4)
- faktorial(4) memanggil faktorial(3)
- faktorial(3) memanggil faktorial(2)
- faktorial(2) memanggil faktorial(1)
- faktorial(1) memenuhi Base Case sehingga mengembalikan nilai 1

Setelah itu hasil akan dihitung kembali secara berurutan:

- $2 \times 1 = 2$
- $3 \times 2 = 6$
- $4 \times 6 = 24$
- $5 \times 24 = 120$

Sehingga hasil Outputnya adalah 120.

KARAKTERISTIK FUNGSI REKURSIF

Fungsi rekursif memiliki beberapa karakteristik utama, yaitu:

1. Fungsi memanggil dirinya sendiri.
2. Memiliki kondisi penghentian atau Base Case.

3. Memiliki proses pengulangan atau Recursive Case.
4. Digunakan untuk menyelesaikan masalah yang memiliki pola berulang.
5. Membutuhkan memori lebih besar dibanding perulangan biasa karena menggunakan stack pemanggilan fungsi.

Karakteristik tersebut membuat rekursif menjadi metode yang unik dalam pemrograman.

15.3.2 Kelebihan dan Kekurangan

Fungsi rekursif memiliki beberapa kelebihan dan kekurangan dalam penggunaannya pada pemrograman. Rekursif sering digunakan untuk menyelesaikan masalah yang memiliki pola berulang karena dapat membuat kode program lebih sederhana.

KELEBIHAN REKURSIF

1. Kode lebih singkat dan rapi Rekursif

membuat penulisan program menjadi lebih sederhana dan elegan dibanding menggunakan banyak perulangan.

2. Mempermudah penyelesaian masalah kompleks

Masalah yang besar dapat dipecah menjadi bagian-bagian kecil sehingga lebih mudah dipahami dan diselesaikan.

KEKURANGAN REKURSIF

1. Menggunakan memori lebih besar

Setiap pemanggilan fungsi rekursif membutuhkan tambahan memori sehingga penggunaan memori menjadi lebih banyak.

2. Kurang efisien dan lebih lambat

Rekursif membutuhkan proses pemanggilan fungsi berulang yang dapat mempengaruhi kecepatan program.

3. Sulit dipahami dan di-debug

Alur program rekursif terkadang sulit dipahami, terutama bagi pemula, dan lebih sulit mencari kesalahan pada program.

15.3.3 Bentuk Umum dan Studi Kasus

Bentuk umum fungsi rekursif pada Python:

```
def function_name(parameter_list):  
    ...  
    function_name(...)  
    ...
```

Sebenarnya semua fungsi rekursif memiliki solusi iteratifnya. Contohnya pada kasus faktorial berikut: Faktorial adalah menghitung perkalian deret angka $1 \times 2 \times 3 \times \dots \times n$. Algoritma untuk menghitung faktorial adalah:

1. Tanyakan n
2. Siapkan variabel total untuk menampung hasil perkalian faktorial dan set nilai awal dengan 0
3. Loop dari $i = 1$ hingga n untuk mengerjakan:
4. $\text{total} = \text{total} * i$
5. Tampilkan total

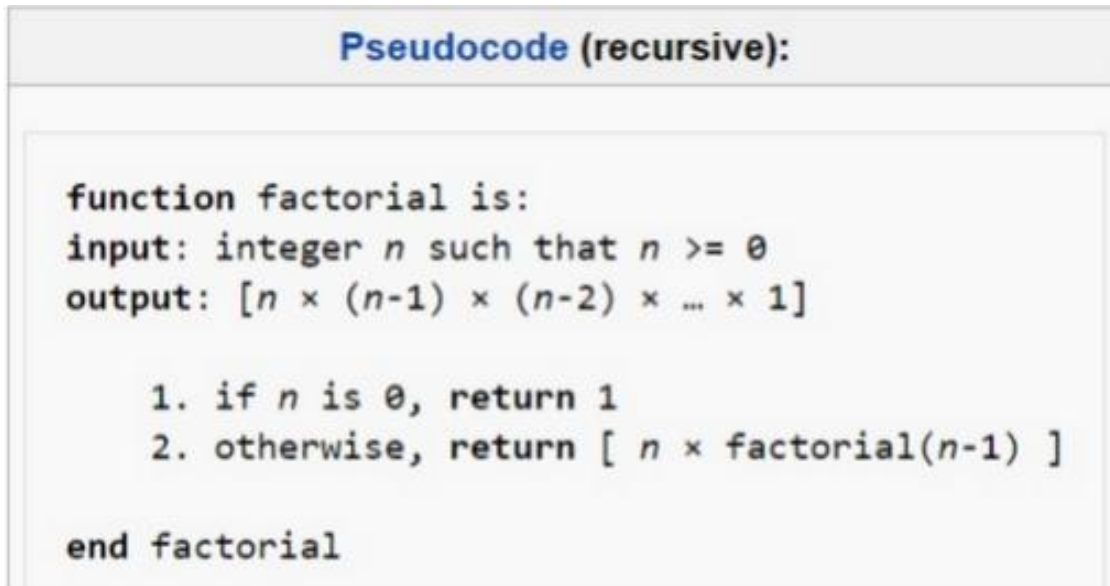
Dengan menggunakan fungsi rekursif maka faktorial bisa dihitung dengan memakai rumus pada gambar 15.1.

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{if } n > 0 \end{cases}$$

Gambar 15.1: Rumus Faktorial. Sumber Modul PrakAlpro

Dari rumus pada Gambar 15.1, dapat dibuat pseudocode secara rekursif seperti di gambar 15.2

Dari pseudocode, maka kode Python yang dapat dibuat seperti di bawah ini:



Gambar 15.2: Pseudocode rekursif faktorial. Sumber Modul PrakAlpro

Programnya seperti di bawah ini:

```
def faktorial(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return faktorial(n-1) * n  
print(faktorial(4))
```

Output dari program tersebut adalah 24

Untuk proses perhitungannya seperti pada Gambar 15.3:

```
1.   
2. calc_factorial(4)           # 1st call with 4  
3. 4 * calc_factorial(3)      # 2nd call with 3  
4. 4 * 3 * calc_factorial(2)  # 3rd call with 2  
5. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1  
6. 4 * 3 * 2 * 1             # return from 4th call as number=1  
7. 4 * 3 * 2                 # return from 3rd call  
8. 4 * 6                     # return from 2nd call  
9. 24                        # return from 1st call
```

Gambar 15.3: Proses perhitungan faktorial rekursif. Sumber Modul PrakAlpro

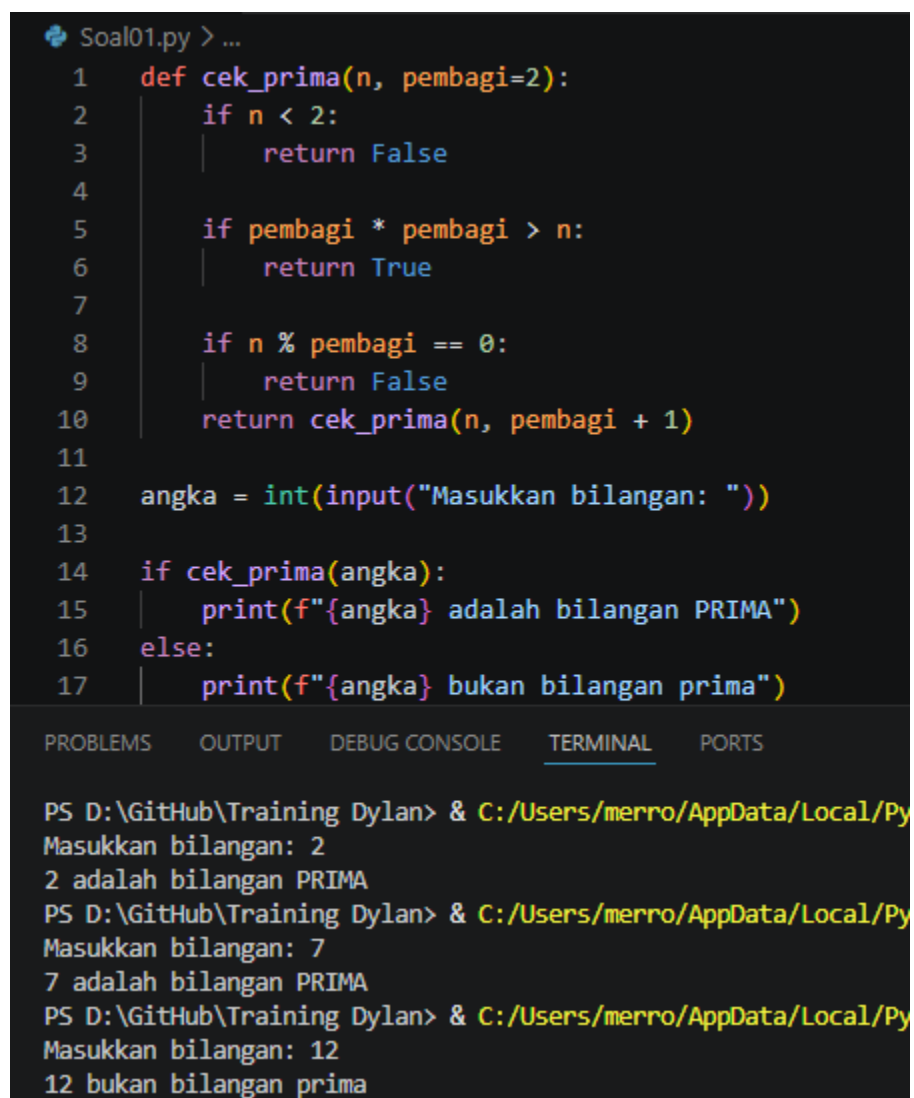
BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github = https://github.com/MichaelDylan-ti/71251182_michaeldylan/tree/main

LATIHAN 15.1

Vidi adalah adik Tono yang sedang belajar bilangan prima. Vidi mengalami kesulitan untuk menentukan suatu bilangan bilangan prima atau bukan. Untuk membantu adiknya Tono kemudian membuat program untuk pengecekan bilangan prima dengan menggunakan fungsi rekursif. Bantulah Tono untuk menyelesaikan tugas tersebut.

JAWABAN:



```
Soal01.py > ...
1  def cek_prima(n, pembagi=2):
2      if n < 2:
3          return False
4
5      if pembagi * pembagi > n:
6          return True
7
8      if n % pembagi == 0:
9          return False
10     return cek_prima(n, pembagi + 1)
11
12     angka = int(input("Masukkan bilangan: "))
13
14     if cek_prima(angka):
15         print(f"{angka} adalah bilangan PRIMA")
16     else:
17         print(f"{angka} bukan bilangan prima")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Py
Masukkan bilangan: 2
2 adalah bilangan PRIMA
PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Py
Masukkan bilangan: 7
7 adalah bilangan PRIMA
PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Py
Masukkan bilangan: 12
12 bukan bilangan prima
```

Gambar 15.4: Input dan Output program dari Soal 15.1. Sumber Python Vscode

Program di atas digunakan untuk mengecek apakah suatu bilangan termasuk bilangan prima atau bukan dengan menggunakan fungsi rekursif. Program dimulai dengan fungsi cek_prima(n, pembagi=2). Variabel n merupakan angka yang akan dicek, sedangkan pembagi digunakan sebagai angka pembagi awal yang dimulai dari 2.

Pertama, program akan memeriksa apakah nilai n kurang dari 2. Jika iya, maka program langsung mengembalikan nilai False karena bilangan kurang dari 2 bukan bilangan prima.

Selanjutnya, program mengecek kondisi $\text{pembagi} * \text{pembagi} > n$. Jika kondisi ini terpenuhi, berarti tidak ada angka lain yang bisa membagi n, sehingga bilangan tersebut dinyatakan prima dan program mengembalikan nilai True. Kemudian program akan memeriksa apakah n habis dibagi oleh pembagi menggunakan operasi modulus %. Jika hasilnya 0, berarti bilangan tersebut bukan prima sehingga program mengembalikan nilai False.

Jika belum memenuhi semua kondisi di atas, fungsi akan memanggil dirinya sendiri dengan nilai $\text{pembagi} + 1$. Proses ini akan terus berulang sampai ditemukan hasil apakah bilangan prima atau bukan. Pada bagian input, pengguna diminta memasukkan sebuah bilangan melalui keyboard. Setelah itu program menjalankan fungsi cek_prima() untuk mengecek bilangan tersebut. Output program akan menampilkan:

- “bilangan PRIMA” jika angka termasuk bilangan prima.
- “bukan bilangan prima” jika angka bukan bilangan prima.

Contohnya, jika input 2, maka outputnya adalah 2 adalah bilangan PRIMA. Sedangkan jika input 12, maka outputnya adalah 12 bukan bilangan prima.

LATIHAN 15.2

Buatlah fungsi rekursif mengetahui suatu kalimat adalah palindrom atau bukan!

JAWABAN:

```
Soal02.py > ...
1  def cek_palindrom(kalimat):
2      kalimat = kalimat.replace(" ", "").lower()
3      def rekursif(s):
4          if len(s) <= 1:
5              return True
6
7          if s[0] != s[-1]:
8              return False
9          return rekursif(s[1:-1])
10     return rekursif(kalimat)
11
12     kalimat = input("Masukkan kata/kalimat: ")
13
14     if cek_palindrom(kalimat):
15         print(f'"{kalimat}" adalah PALINDROM')
16     else:
17         print(f'"{kalimat}" bukan palindrom')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Python/Python38-64/Python.exe
Masukkan kata/kalimat: Ada
"Ada" adalah PALINDROM
PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Python/Python38-64/Python.exe
Masukkan kata/kalimat: Katak
"Katak" adalah PALINDROM
PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Python/Python38-64/Python.exe
Masukkan kata/kalimat: Python
"Python" bukan palindrom
```

Gambar 15.5: Input dan Output program dari Soal 15.2. Sumber Python Vscode

Program di atas digunakan untuk mengecek apakah sebuah kata atau kalimat termasuk palindrom atau bukan dengan menggunakan fungsi rekursif. Palindrom adalah kata atau kalimat yang tetap sama jika dibaca dari depan maupun belakang. Contohnya seperti “Ada” dan “Katak”.

Pada program, fungsi `cek_palindrom(kalimat)` digunakan untuk memeriksa kata atau kalimat yang diinput pengguna. Pertama, program akan menghapus spasi menggunakan `replace(" ", "")` dan mengubah semua huruf menjadi huruf kecil dengan `lower()`. Tujuannya agar pengecekan lebih akurat dan tidak membedakan huruf besar maupun kecil. Di dalam fungsi tersebut terdapat

fungsi rekursif rekursif(s) yang bekerja dengan membandingkan huruf pertama dan huruf terakhir.

- Jika panjang teks kurang dari atau sama dengan 1, program mengembalikan nilai True karena kata tersebut dianggap palindrom.
- Jika huruf pertama dan terakhir berbeda, program mengembalikan nilai False.
- Jika sama, fungsi akan memanggil dirinya sendiri untuk mengecek bagian tengah kata sampai semua huruf selesai diperiksa.

Pada bagian input, pengguna diminta memasukkan kata atau kalimat. Setelah itu program akan menampilkan hasil apakah kata tersebut palindrom atau bukan.

Contoh input dan output:

- Input: "Ada"
Output: "Ada" adalah PALINDROM
- Input: "Katak"
Output: "Katak" adalah PALINDROM
- Input: "Python"
Output: "Python" bukan palindrom

LATIHAN 15.3

Buatlah fungsi rekursif untuk menghitung jumlah deret ganjil dari $1 + 3 + 5 + \dots + n$!

JAWABAN:

(Di halaman berikutnya :v)

```
Soal03.py > ...
1  def suku(n):
2      return (2 ** n) - 1
3
4  def deret_ganjil(n):
5      if n == 1:
6          return 1
7      return suku(n) + deret_ganjil(n - 1)
8
9  def tampilkan_deret(n):
10     if n == 1:
11         print(suku(1), end="")
12     else:
13         tampilkan_deret(n - 1)
14         print(f" + {suku(n)}", end="")
15
16  n = int(input("Masukkan jumlah suku (n): "))
17  print("Deret  : ", end="")
18  tampilkan_deret(n)
19  print(f"\nJumlah : {deret_ganjil(n)}")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local
Masukkan jumlah suku (n): 7
Deret  : 1 + 3 + 7 + 15 + 31 + 63 + 127
Jumlah : 247
```

Gambar 15.6: Input dan Output program dari Soal 15.3. Sumber Python Vscode

Program di atas digunakan untuk menampilkan deret bilangan ganjil dan menghitung jumlah total deret tersebut menggunakan fungsi rekursif. Fungsi `suku(n)` digunakan untuk mencari nilai suku ke- n dengan rumus $(2^n) - 1$. Rumus tersebut menghasilkan bilangan ganjil seperti 1, 3, 7, 15, dan seterusnya.

Fungsi `deret_ganjil(n)` berfungsi untuk menghitung jumlah seluruh suku secara rekursif. Jika $n == 1$, program akan mengembalikan nilai 1 sebagai *Base Case*. Jika tidak, fungsi akan menjumlahkan suku ke- n dengan hasil dari pemanggilan fungsi sebelumnya sampai mencapai kondisi berhenti. Sedangkan fungsi `tampilkan_deret(n)` digunakan untuk menampilkan deret angka satu per satu secara berurutan menggunakan rekursif. Pada bagian input, pengguna diminta memasukkan jumlah suku (n). Pada contoh, input yang digunakan adalah 7.

Maka deret yang ditampilkan adalah:

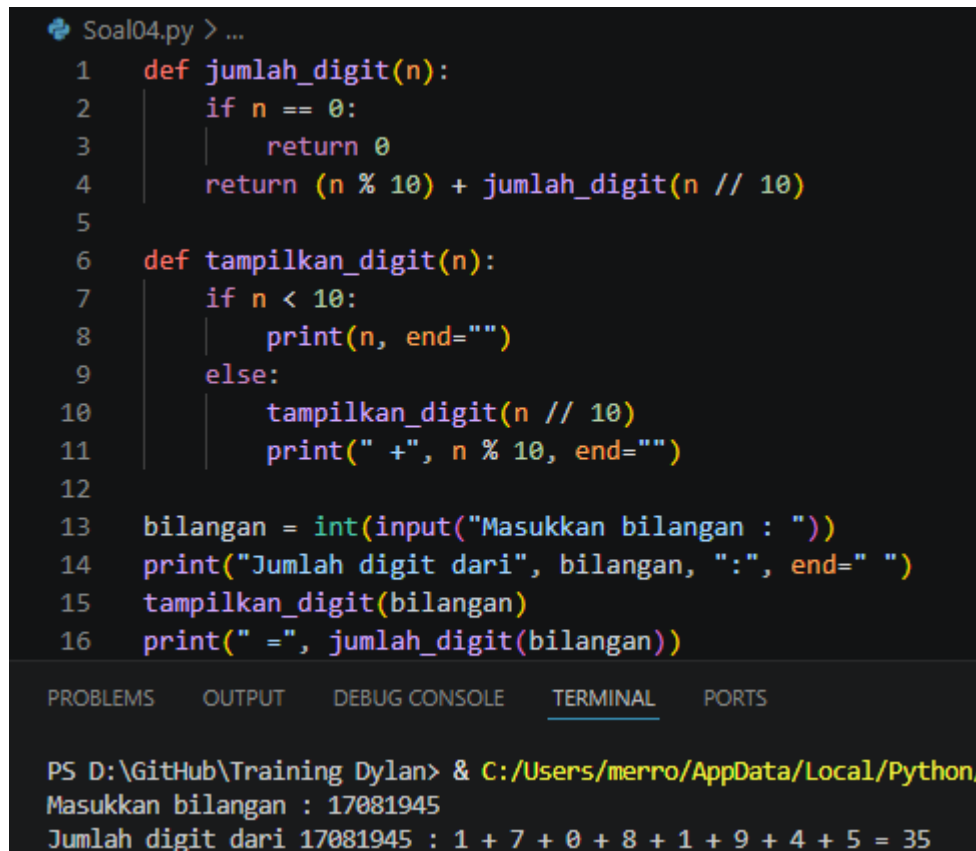
1 + 3 + 7 + 15 + 31 + 63 + 127

Kemudian program menghitung jumlah seluruh deret tersebut sehingga menghasilkan 247
Jadi output program adalah menampilkan deret bilangan ganjil sebanyak 7 suku beserta jumlah total dari seluruh deret tersebut.

LATIHAN 15.4

Buatlah fungsi rekursif untuk mengetahui jumlah digit dari suatu bilangan. Seperti misalnya tulisan: "234" maka jumlah digitnya adalah $2+3+4 = 9$!

JAWABAN:



```
Soal04.py > ...
1  def jumlah_digit(n):
2      if n == 0:
3          return 0
4      return (n % 10) + jumlah_digit(n // 10)
5
6  def tampilkan_digit(n):
7      if n < 10:
8          print(n, end="")
9      else:
10         tampilkan_digit(n // 10)
11         print(" +", n % 10, end="")
12
13  bilangan = int(input("Masukkan bilangan : "))
14  print("Jumlah digit dari", bilangan, ":", end=" ")
15  tampilkan_digit(bilangan)
16  print(" =", jumlah_digit(bilangan))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/
Masukkan bilangan : 17081945
Jumlah digit dari 17081945 : 1 + 7 + 0 + 8 + 1 + 9 + 4 + 5 = 35
```

Gambar 15.7: Input dan Output program dari Soal 15.4. Sumber Python Vscode

Program di atas digunakan untuk menghitung jumlah seluruh digit dari sebuah bilangan menggunakan fungsi rekursif. Fungsi `jumlah_digit(n)` berfungsi untuk menjumlahkan setiap digit

pada bilangan. Jika nilai n sama dengan 0, fungsi akan berhenti dan mengembalikan nilai 0 sebagai *Base Case*. Jika tidak, program akan mengambil digit terakhir menggunakan $n \% 10$, lalu menambahkannya dengan hasil pemanggilan fungsi rekursif dari $n // 10$ sampai semua digit habis.

Sedangkan fungsi `tampilkan_digit(n)` digunakan untuk menampilkan setiap digit dalam bentuk penjumlahan secara berurutan. Pada bagian input, pengguna diminta memasukkan sebuah bilangan. Pada contoh, input yang digunakan adalah 17081945.

Maka program akan menampilkan proses penjumlahan digit seperti berikut:

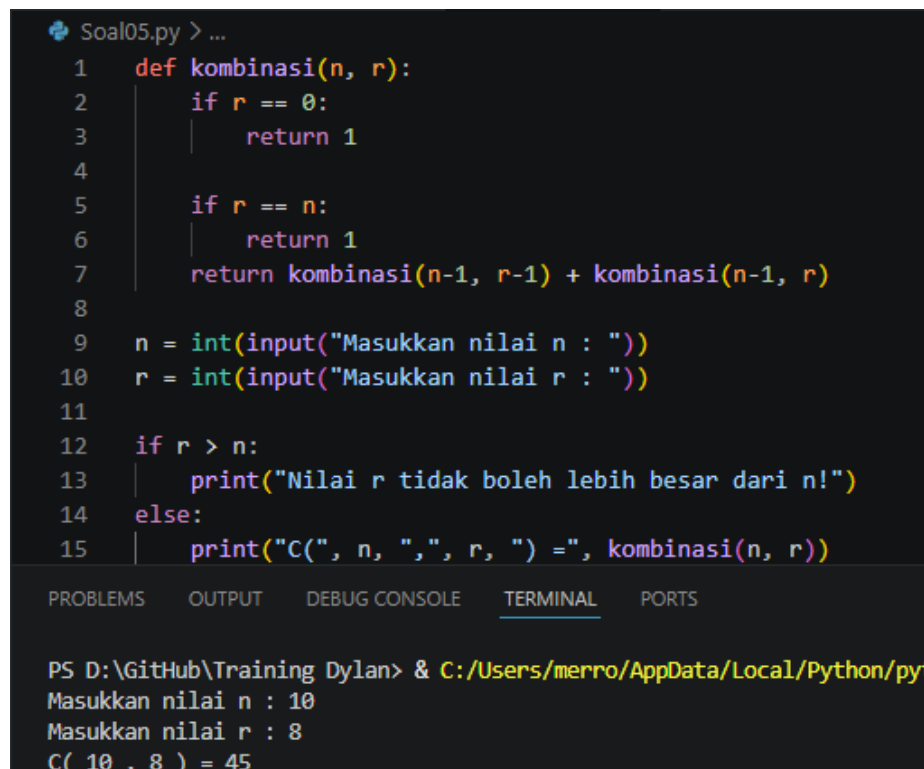
$1 + 7 + 0 + 8 + 1 + 9 + 4 + 5$

Kemudian seluruh digit dijumlahkan sehingga menghasilkan 35. Jadi output program menunjukkan penjumlahan setiap digit dari bilangan yang dimasukkan beserta hasil totalnya.

LATIHAN 15.5

Buatlah fungsi rekursif untuk menghitung kombinasi!

JAWABAN:



```
Soal05.py > ...
1  def kombinasi(n, r):
2      if r == 0:
3          return 1
4
5      if r == n:
6          return 1
7      return kombinasi(n-1, r-1) + kombinasi(n-1, r)
8
9  n = int(input("Masukkan nilai n : "))
10 r = int(input("Masukkan nilai r : "))
11
12 if r > n:
13     print("Nilai r tidak boleh lebih besar dari n!")
14 else:
15     print("C(", n, ", ", r, ") =", kombinasi(n, r))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Python/py
Masukkan nilai n : 10
Masukkan nilai r : 8
C( 10 , 8 ) = 45
```

Gambar 15.8: Input dan Output program dari Soal 15.5. Sumber Python Vscode

Program di atas digunakan untuk menghitung nilai kombinasi $C(n, r)$ menggunakan fungsi rekursif. Kombinasi biasanya digunakan untuk mencari banyak cara memilih sejumlah objek tanpa memperhatikan urutan. Fungsi kombinasi(n, r) bekerja dengan konsep rekursif. Jika nilai $r == 0$ atau $r == n$, fungsi akan mengembalikan nilai 1 sebagai *Base Case*. Hal ini karena dalam kombinasi, memilih 0 objek atau memilih seluruh objek hanya memiliki 1 kemungkinan. Jika kondisi tersebut belum terpenuhi, fungsi akan memanggil dirinya sendiri dengan rumus.

- kombinasi($n-1, r-1$)

- kombinasi($n-1, r$)

Lalu kedua hasil tersebut dijumlahkan untuk mendapatkan nilai kombinasi. Pada bagian input, pengguna diminta memasukkan nilai n dan r . Program juga akan mengecek apakah r lebih besar dari n . Jika iya, program menampilkan pesan error karena nilai r tidak boleh lebih besar dari n .

Pada contoh:

$n = 10$

$r = 8$

Maka program akan menghitung nilai kombinasi $C(10,8)$ dan menghasilkan 45

Jadi output program adalah $C(10, 8) = 45$. Program ini menunjukkan cara penggunaan rekursif untuk menghitung kombinasi berdasarkan rumus matematika kombinasi.